Systemsoftware WiSe 2026/27 – Übung 1

Name: Justin Reider Matrikelnummer: 1751840

Aufgabe 1

Programm: Ein Programm ist eine passive Folge von Anweisungen, die als Datei gespeichert ist. Wird es ausgeführt, entsteht daraus ein Prozess.

Prozess: Ein Prozess ist ein Programm in Ausführung. Er hat einen eigenen Adressraum und eigene Ressourcen (Speicher, geöffnete Dateien) und enthält mindestens einen Thread.

Thread: Ein Thread ist ein Ausführungsstrang innerhalb eines Prozesses. Er hat einen eigenen Befehlszähler und Stack und teilt sich Adressraum und Ressourcen mit den anderen Threads desselben Prozesses.

Aufgabe 2

a) Jedes Programm hat einen sequentiellen Anteil, der nicht parallelisiert werden kann. Er bestimmt die minimale Laufzeit, egal wie viele Kerne es gibt. Dazu kommen Overhead für Erzeugung und Synchronisation der Threads und gemeinsam genutzte Ressourcen wie Speicher und I/O, auf die die Kerne warten müssen. Der Takt lässt sich wegen Leistungsaufnahme und Wärmeentwicklung nicht beliebig erhöhen.

b) Amdahlsches Gesetz: $S = 1 / ((1 - p) + p / n)$

Rechenweg für $p = 0{,}75$ und $n = 8$: $S = 1 / (0{,}25 + 0{,}75/8) = 1 / 0{,}34375 \approx 2{,}91$

p \ n	2	4	8	16
25 %	1,14	1,23	1,28	1,31
50 %	1,33	1,60	1,78	1,88
75 %	1,60	2,29	2,91	3,37

Aufgabe 3

1) Das Java-Programm `Counter` zählt in den Testdaten (2000 Java-Dateien, ca. 10,6 Mio. Zeilen) alle Zeilen in ca. 1630 ms.

2) `java/ParallelCounter.java` sammelt zuerst alle passenden Dateien. Danach verteilt ein Thread-Pool mit n Threads die Dateien auf die Threads, und die Teilsummen werden am Ende addiert. Aufruf: `java ParallelCounter <Ordner> <Regex> <Threads>`

3) Messungen (Ryzen 9 5950X, 16 Kerne / 32 Threads, Samsung 970 EVO SSD, Testdaten 826 MB, Median aus 5 Läufen). Gemessen wurde mit warmem Dateicache, die Daten lagen also bereits im RAM:

Threads	Zeit	Speedup

sequentiell	1634 ms	1,00
1	1704 ms	0,96
2	952 ms	1,72
4	588 ms	2,78
8	469 ms	3,48
16	455 ms	3,59
32	464 ms	3,52
64	ca. 1400–2300 ms	≈ 1

Beobachtungen: Bis 8 Threads wird das Programm deutlich schneller, aber nicht linear. Ab 8 Threads bleibt der Speedup bei etwa 3,5, weil das Sammeln der Dateien sequentiell ist und sich alle Threads Speicher und Dateisystem teilen. Mit mehr Threads als logischen Kernen (64) wird es wieder langsamer, weil die Threads um die Kerne konkurrieren und mehr Overhead entsteht.